Answer the following questions in detail. Your answer should be well structured and clearly explained. Feel free to cite references where applicable.

1. Use aggregate analysis to determine the amortized cost per operation for a sequence of n operations on a data structure in which the i th operation costs i if i is an exact power of 2, and 1 otherwise.

The question asks that

cost = i [if i is exact power of 2]

cost = 1 [else]

To get amortized cost, we need to calculate total cost over n operation & divide by n, to get the average.

Now, power of 2 are

$1, 2, 4, \ldots 2^{\lfloor n}$

Now, since power of 2 comes rarely between 1 & n, there are about $\log_2(n)$ power of 2.

Here we have to consider,

All non power of 2 operations cost 1. We can say they contribute rougly n of the total cost. While for the power of 2 operations cost up to largest powe of 2 less than n. So, in total their value is less than 2n.

So, the total cost should be $< 3n$ [non-power being close to n & power being less than 2n]

which is $O(n)$

& now diviaing by n operation: $O(n)/n = O(1)$

So, the amortized cost per operation is $O(1)$

slides

2. You perform a sequence of PUSH and POP operations on a stack whose size never exceeds k. After every k operations, a copy of the entire stack is made automatically, for backup purposes. Show that the cost of n stack operations, including copying the stack, is $O(n)$ by assigning suitable amortized costs to the various stack operations. [Use the accounting method]

So we are given

The stack size $\leq k$

Copy cost $= O(k)$

Total operation $= n$

& Full stack copy after every k operations.

Using the accounting method,

We assign an amortized cost $= 2$, to every Push & Pop operation

where, 1 pays actual cost

1 is saved as credit

So, after 'k' operation, we saved up 'k' credits.

When full stack copy occurs, after 'k' operation, we use the saved credits to pay for the copy cost.

Since, this is repeated every k operation, the total operation cost, n grows linearly with n.

So, including copying, the total cost of n operation is

$O(n)$ //

~~$O(n)$~~

3. Consider an ordinary binary min-heap data structure supporting the instructions INSERT and EXTRACT-MIN that, when there are n items in the heap, implements each operation in $O(\log n)$ worst-case time. Give a potential function ϕ such that the amortized cost of i NSERT is $O(\log n)$ and the amortized cost of EXTRACT-MIN is $O(1)$, and show that your potential function yields these amortized time bounds. Note that in the analysis, n is the number of items currently in the heap, and you do not know a bound on the maximum number of items that can ever be stored in the heap.

We need a potential function ϕ, where the amortized cost of

INSERT = $O(\log n)$

EXTRACT-MIN = $O(1)$

But the operation in a Binary Min-Heap is,

EXTRACT-MIN = $O(\log n)$

Now we define a potential function ϕ that depends on no. of elements in the heap. So, we assign potential proportional to $n \log n$,

When we perform INSERT, the heap size & potential both increases,

So, the amortized cost remains $O(\log n)$.

$$O(\log n) \Rightarrow \phi(n+1) - \phi(n) = (n+1)\log(n+1) - n\log n = O(\log n)$$

When we perform EXTRACT-MIN, heap size & potential both decreases,

Let's say there is a constant c, which cancels out $O(\log n)$ work in potential.

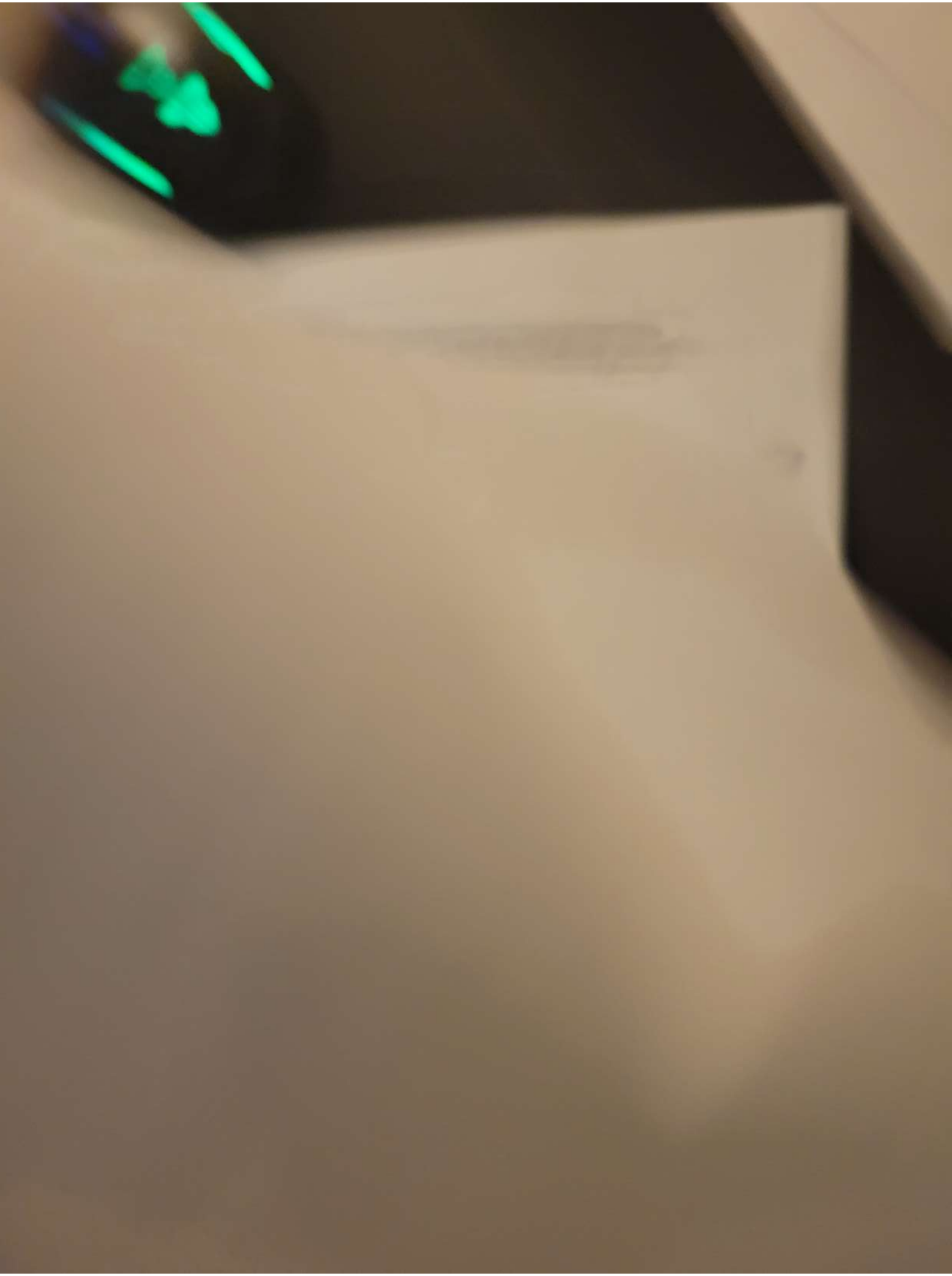
So, $\phi = c \cdot n \log n$

From actual cost to potential decrease

$$O(\log n) \Rightarrow \phi(n) - \phi(n-1) = c[n\log n - (n-1)\log(n-1)] = O(\log n)$$

Here, let's choose c large enough so potential drop cancel actual cost

So, amortized cost = $O(1)$



The potential function is

$$\emptyset(H) = c n \log n$$

4. In a priority queue implemented using a binary heap, some operations (like insertions and deletions) might take longer when the heap needs to be restructured. Explain, in words, how amortized analysis is used to show that the average cost per operation (insert, delete) over a sequence of operations remains efficient, despite occasional restructuring of the heap.

→ In a binary heap, insertions & deletition might need restructuring like insert may bubble up an element while delete might bubble down elements. With the worst case senario-cost., it can take O(log n) time ~~but~~ because some operation might ~~not~~ use many swaps. But, Amortized analysis shows that not every operation trigger worst-case restructing. Also the expensive operation are balanced by cheaper ones & over many operation, the total restructuring cost grow proportionaly to the total no. of operation. So, some operation might look expensive at one point, but what is need to be seen is the average cost per operation & armortized analysis helps to visualize that some data structure is effecient over time.

5. Show the amortized analysis of Binary Counter implantation from our class slides using aggregate, accounting, and potential methods.

~~Aggregate Method~~

Lets use 0111 +1 → 1000

This flips 4 bits

Worst-case cost of one increment = O(k)

Worst-case cost of n increment = O(nk)

Aggregate method :-

The least significant bit flips every increment, 2nd bit every 2 increment, 3rd every 4 increment. So, in general, bit k is fliped every 2^k increment.

So, over n-increments, bit fliped is:

0 = n

1 = n/2

2 = n/4

i = $n/2^i$

So, total flips = $n + n/2 + n/4 + \cdots n/2^i$

Here, it is a geometric series with sum less than 2n

So, total cost of n increments = O(n)

&

Amortized cost per increment: O(n)/n

=) O(1)

2) Accounting method

No. of charge per increment = 2

When incrementing 0, 1 unit pays for setting the rightmost 0 to 1 & 1 unit is saved as credit on that bit.

Later, when the bit flips from 1 to 0, it uses the saved credit for flip.

So, no operation runs out of credit.

∴ Total amortized cost per increment:

$$2 = O(1).$$

So, the total cost over n increment $= O(n)$

3) Potential Method

Let ϕ = no. of 1s in the counter

It measures no. of flips from 1 to 0 that may occur.

Considering one increment, suppose t trailing 1s flip to 0 & one '0' flips to '1'. Taking the cost to

$$t = 1$$

Change in potential is that we lose t ones & gain 1 one

So,

$$\Delta\phi = 1 - t$$

Amortized cost = Actual cost $+ \Delta\phi = t + 1 + 1 - t = 2$

So, Amortized cost $= 2 = O(1)$

6. A splay tree is a self-adjusting binary search tree in which each operation (insert, delete, search) involves a process called "splaying," which moves the accessed node to the root. Describe how amortized analysis helps demonstrate that the amortized time complexity of a series of operations on a splay tree can be bounded by $O(logn)$, even though some individual operations may take longer. Show the proof of this for the ZagZig rotation

→ We know that amortized analysis basically shows that ~~over~~ average ~~sequence of operations, the avera~~ cost of operation. A splay tree moves accessed node to root using rotation & some operation can take $O(n)$ time in worst case. But, for amortized analysis the average cost is $O(\log n)$.

Choosing potential function

$S(x)$: subtree size of node x

$r(x) = \log_2 S(x)$ = rank of node x

Potential: $\Phi = \sum_x r(x)$

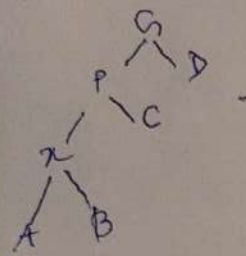
This potential function represents the structural ~~imp~~ imbalance of the tree.

So, the amortized cost = Actual cost + Change in potential ($\Delta\Phi$), which results if potential decreases significantly during restructure, that decrease helps pay for the rotation

2) Accounting method

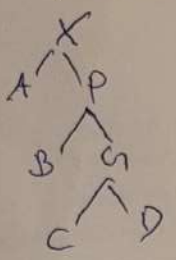
No. of cha...

Initial rank (r)



Splay(x)

Final rank(r')



g the

uses

Here, Zig2ag occured when x is a left child of p & p is right child of g. So, two rotation were performed, x with p & x with g.

That results on constant cost $\doteq O(1)$

So, only x, p, g change ranks where x moves up & p & g move down,

~~Amortized~~

Amortized cost = Actual cost + $\Delta\phi$

for zig-zig, it can be bounded by:

$$O(1) + 3(r'(x) - r(x))$$

During a full splay, these rank increases:

$$r' - r$$

Since, $r(x) = \log s(x) \leq \log n$,

The total amortized cost of a splay is

$$O(\log n)$$